

Team Members: Caroline Locke, Chaitanya Irkar
Andrew IDs: chlocke, cirkar

1. Describe your design in terms of organization and operation.

Our design is a 13 stage Riscv pipeline implementation split into 2 fetch stages, a decode stage, an execute stage, 8 memory stages, and a writeback stage. The following bullet points provide an overview of each stage. For a more detailed look at the implementation of the pipeline, a datapath is shown in pipeline.pdf.

- Fetch 1: In the fetch stage, the program counter is used to calculate the instruction address and requests the instruction from memory. Furthermore, an sram with 1 read and 1 write port is used to predict branching via an always not taken prediction strategy.
- Fetch 2: The instruction memory latency is 2 cycles, so the fetch 2 stage represents this memory delay.
- Decode: In the decode stage, the instruction is decoded
- Execute: The execute stage performs any computation determined by the instruction. If necessary any data is forwarded here, and these values are then supplied to the ALU to perform its operations. Furthermore, if the instruction being processed currently is a branching instruction, the execute stage determines if the branch was correctly or incorrectly predicted in the fetch stage, and updates the BTB table data to be sent back to the fetch 1 stage to be written back. Lastly, if the branch was incorrectly predicted then the execute stage alerts the decode stage that the previous instructions will need to be flushed.
- Memory 1: The memory 1 stage handles all memory accesses. If an instruction is a load instruction, then it checks the cache to see if the requested data can be quickly retrieved, and otherwise requests this data from main memory. If the data was in the cache, then based on the type of load instruction, it determines which bytes of data are being asked for as well. If the instruction is a store instruction, then this data is written straight to the cache and at the same time also written back to main memory. Lastly, with the implementation of a store buffer, if the instruction being processed does not read or write to memory, and the size of the store buffer is not 0, then that data is written to the cache.
- Memory 2-Memory 8: As data memory delay has a latency of 8 cycles, Memory 2-Memory 8 stages propagate any necessary signals to the writeback stage. If the instruction is a load instruction that had a cache hit or a non-load or store instruction then the data in these stages can be forwarded back to later instructions in previous stages, otherwise the pipeline will stall until the necessary data has been retrieved.
- Writeback: Once an instruction reaches the writeback stage, if it is a load instruction and has a cache miss in the Memory 1 stage, then this data has been loaded from memory. If any data needs to be written back to the register file, then this information is passed back to the decode stage in order to be written. This same data will also be sent back to the Memory 1 stage to be written back to the cache. Lastly, if the instruction is an ECALL instruction, once it reaches the writeback stage, then this instruction will begin to be handled.

We split our implementation into separate modules based on the stage, with each stage separated by registers to propagate all signals. Outside of each of these modules, we also include stalling logic for any hazards, and forwarding signals that determine if a signal needs to be forwarded and from what stage.

2. Describe the critical path of your synthesized implementation. (Is it different than what you expected?) How does the clock frequency compare to your Lab 3 implementation?

The critical path in our design is highlighted in the datapath along with relevant delays in pipeline.pdf. It occurs when an instruction needs to be forwarded into the execute stage, this forwarded value is used to compute alu_out, and the branch is shown to have been mispredicted. Then a signal, needToFlush is enabled which is then used to determine the correct next pc and the instr_addr is updated accordingly.

Clock Frequency in Lab 4 Design: 5.60 ns

Clock Frequency in Lab 3 Design: 4.23 ns

Our critical path is higher in our lab 4 design compared to in our lab 3 design. This is likely due to the additional forwarding paths due to the longer memory delay and additional pipeline stages since the critical path follows the same path in both implementations.

3. If you kept the cache, describe how you selected the parameters. If you made any modifications, mention them here along with reasoning. (Replacement policy, capacity, ways, etc.) If you removed the cache, provide reasoning for why.

We decided to keep the cache, since the delay to load data from main memory is so large. We did not modify the cache447 module, and instead focused on sweeping the parameters used in order to optimize the performance of our pipeline. We gathered the following data presented in the tables below, and drew the following conclusions.

Policy = 1 (LRU) , BTB Strategy = 0

With Store Buffer:

Index Bits	Ways	IPCs- gaussian.c	IPC- max_path_sum.c	IPC- transpose.c	IPC- quicksort.c
2	2	0.624215	0.643957	0.459499	0.620248
	4	0.721713	0.652526	0.459499	0.637445
	8	0.727365	0.666205	0.459499	0.653040
3	2	0.665175	0.649379	0.459499	0.638617
	4	0.726132	0.662815	0.460010	0.652400

	8	0.777226	0.698172	0.573201	0.700456
4	2	0.691919	0.661050	0.461893	0.655126
	4	0.776991	0.698602	0.586745	0.707381
	8	0.777226	0.737240	0.618301	0.715869
5	2	0.772144	0.699217	0.594328	0.710698
	4	0.777226	0.742061	0.618301	0.715869
	8	0.777405	0.753462	0.618532	0.715869
6	2	0.775818	0.736967	0.618301	0.715869
	4	0.777405	0.753462	0.618532	0.715869
	8	0.806591	0.753462	0.618532	0.715869

Findings:

- It seems that the difference from increasing Ways from 4->8 has little impact on increasing IPC, and likely will not be worth the performance/power tradeoff

In order to explore how power scales with cache size:

Make synth CLOCK_PERIOD = 5.7

Index Bits	Ways	Period (ns)	Power (mW)	MIPS (gaussian.c)	MIPS (max_path_sum.c)	MIPS (transpose.c)	MIPS (quicksort.c)	Average MIPS	MIPS/Watt
2	4	5.7	157.9867	126.616	114.478	80.614	111.832	108.385	686.039
4	2	5.7	130.1080	121.389	115.974	81.034	114.934	108.333	832.639
	4	5.7	153.0139	136.314	122.562	102.938	124.102	121.479	793.908
	8	5.7	232.4415	136.355	129.340	108.474	125.591	124.940	537.512
6	4	5.7	239.2427	136.387	132.186	108.514	125.591	125.670	525.280

Findings:

- Power increases as the number of ways increases. This corresponds with a lower MIPS/Watt as Index Bits increase and number of Ways increases.
- We were surprised by the large power consumption when Index bits = 2 and Ways = 4, we reran synthesis and got similar results, but are still unsure as to why this might be.

Based on these results, we ultimately chose to go with Index Bits = 4 and Ways = 2 in order to minimize power with a small drop in MIPS.

- For any other parameters you changed (forwarding type, BTB size, etc.), describe changes and provide your reasoning.

Branch Prediction Strategies:

- 0- Always Not Taken
- 1- Always Taken
- 2- 2 bit Hysteresis Counter

LRU, ways = 4, index bits = 2

BTB Strategy	Num Words In BTB	IPC- gaussian.c	IPC- max_path_sum.c	IPC- transpose.c	IPC- quicksort.c
0	128	0.721713	0.652526	0.459499	0.637445
1	128	0.738603	0.711933	0.532772	0.784598
2	128	0.780926	0.751523	0.535356	0.792645

BTB Strategy	Num Words In BTB	Period (ns)	Power	MIPS-gaussian.c	MIPS-max_path_sum.c	MIPS-transpose.c	MIPS-quicksort.c	Avg MIPS	MIPS/Watt
0	0	5.68	124.0682 mW	127.062	114.881	80.898	112.226	108.767	876.669
1	128	5.54	160.2715 mW	133.322	128.508	96.168	141.624	124.905	779.334
2	128	5.72	155.7976 mW	136.526	131.385	93.594	138.574	125.020	802.450

Findings:

- In terms of MIPS and MIPS/Watt, the 2-bit hysteresis counter outperforms the branch-always taken prediction strategy .
- The 2-bit hysteresis counter outperforms branch always not taken in terms of MIPS, but branch always not taken outperforms the 2-bit hysteresis counter in terms of MIPS/Watt.
 - It would be interesting to look into the critical path for branch not taken to see if any of this logic could be optimized now that the BTB is no longer being used in order to increase MIPS

Ultimately we chose to use an always not taken prediction strategy in order to minimize our implementation's power consumption- based on this table we thought that that the increase in power consumption and slightly longer critical path for the small increase in IPS.

- For any new features you added, describe them here with reasoning and an approximation of the improvement provided.

We implemented a store buffer, that is used to store data back into the cache after a cache miss for a load instruction. We saw significant improvement in our IPC after implementing this

feature. For example, before implementing this feature, gaussian.c took 182,204 cycles to run, and with the store buffer, this number shrank to around 74,779 cycles.

perf_benchmark	Number of Cycles- Stalling for a Cache Miss	Number of Cycles- With Store Buffer (Approximately)	Improvement
gaussian.c	182,204	74,779	58.96%
transpose.c	29,476	21,617	26.66%
quicksort.c	58,917	50,169	14.85%
max_path_sum.c	33,526	24,350	27.37%

6. Estimate the number of hours spent on
 - a. planning the design in your mind or on paper: 8 hours
 - b. capturing the design (that is, actually hacking in Verilog): 10 hours
 - c. testing the design (not including debug time): 30 hours
 - d. debugging the design (actually fixing the bugs once found): 8 hours
 - e. analyzing the performance of the design: 15 hours